HACKYARD IIT GUWAHATI · PRODUCT REQUIREMENTS DOCUMENT

DeployForge

Instant, self-hosted preview environments for every Git branch.

DeployForge is a zero-configuration preview deployment engine that creates an isolated Docker environment, configures a reverse-proxy route, and gives a developer a shareable preview URL with live resource monitoring.

> **Product promise:** Push a branch, get a working preview environment in seconds, without cloud vendor lock-in or manual port forwarding.

Document **Product Requirements Document**	Version **1.0 · Hackathon MVP**
Backend **Node.js + Express.js**	Database **Local MongoDB in Docker**

This document describes what is implemented, how to run it, how to explain it, and what should be added next.

1. Executive Summary

DeployForge reduces the effort required to share a feature branch with teammates, reviewers, or judges. Instead of manually building an image, finding a port, starting a container, editing Nginx, and sharing a link, the user starts one deployment request. The backend validates the input, reserves a port, starts a resource-limited container through Dockerode, writes a safe Nginx route, stores the deployment state in MongoDB, and exposes status through a React dashboard.

The current MVP is deliberately simple and understandable. It supports an allowlisted Docker image such as `nginx:alpine`, local MongoDB, mock Docker mode for demos, real Docker mode for a Linux server, and generated Nginx configuration files.

2. Problem Statement

- Preview deployments involve repetitive infrastructure commands.
- Port forwarding makes a local feature difficult to share.
- Cloud preview services can create cost and vendor lock-in.
- Teams cannot easily see container health and resource usage.
- Manual cleanup leaves stale containers, ports, and proxy routes.

Opportunity

Build a small self-hosted control plane for fast-moving teams. It does not try to replace Kubernetes; it focuses on a single Linux machine and a clear branch-to-preview workflow that is easy to understand and demonstrate.

3. Goals and Non-Goals

Goals

- Create one preview per active branch.
- Run each preview in an isolated Docker container.
- Reserve ports without collisions.
- Generate predictable preview URLs.
- Display status, CPU, memory, and logs.
- Cleanly stop previews and release resources.
- Protect the server from unsafe input.

Non-goals for MVP

- Multi-region orchestration.
- Production application deployment.
- Automatic autoscaling.
- Enterprise billing or complex RBAC.
- Full CI/CD replacement.
- Running arbitrary untrusted images.

4. Users and User Journey

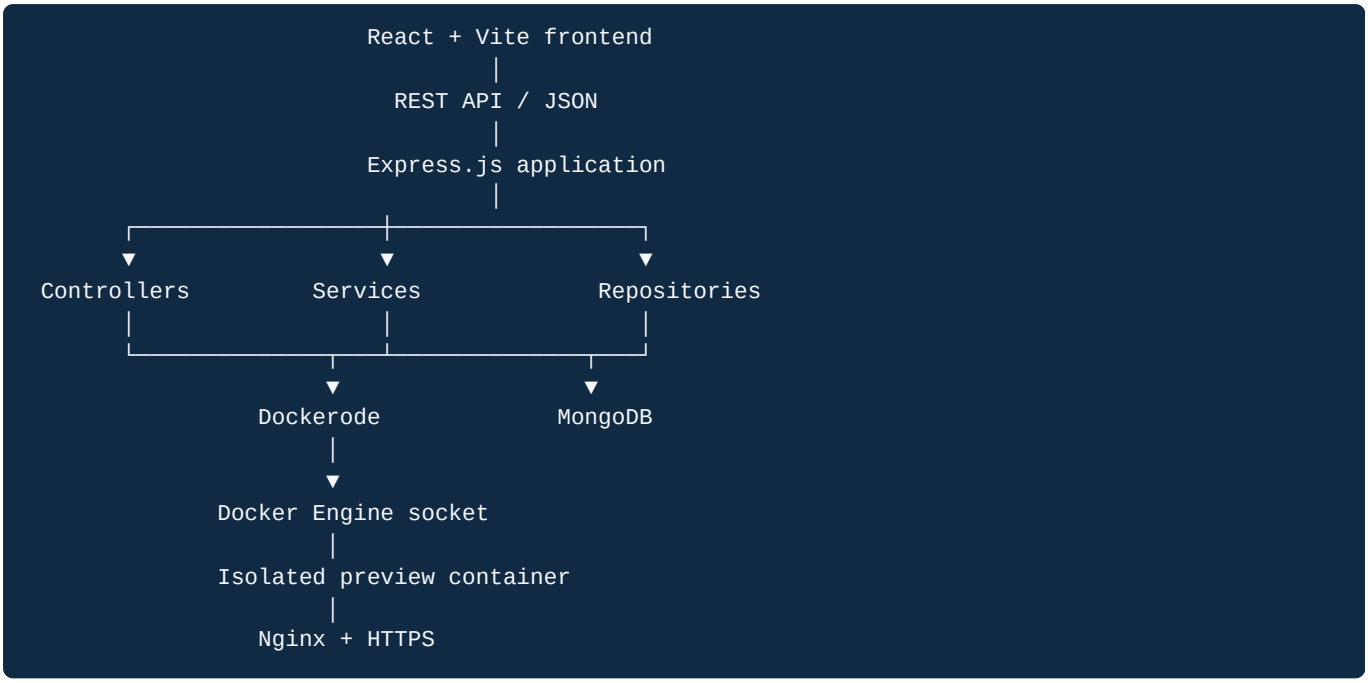
Primary user: a developer who wants to share a feature branch without becoming responsible for infrastructure setup.

```
Developer selects a branch
          ↓
Frontend sends a deployment request
          ↓
Express controller validates the request
          ↓
Service reserves a host port
          ↓
Dockerode starts a limited container
          ↓
Nginx configuration is generated and validated
          ↓
MongoDB stores the running state
          ↓
React dashboard shows the preview and metrics
```

5. Functional Requirements

ID	Requirement	Acceptance criteria
FR-01	Create deployment	Accept a valid branch and allowlisted image and return a deployment record.
FR-02	Port management	Choose an unused port from the configured range and reuse it after cleanup.
FR-03	Container isolation	Start a uniquely named container with labels and CPU/memory limits.
FR-04	Deployment state	Persist status, URL, port, container ID, logs, and timestamps.
FR-05	Proxy route	Write a route file and run `nginx -t` before reload when enabled.
FR-06	Monitoring	Return container CPU, memory, and recent logs.
FR-07	Cleanup	Stop the container, remove its route, mark it stopped, and release its port.
FR-08	Failure handling	Clean partial resources and return a safe error when a deployment fails.
FR-09	Dashboard	Show active previews, status, URL, port, CPU, memory, and delete action.

6. Technical Architecture



The backend follows an enhanced MVC pattern. Controllers translate HTTP requests, services own business workflows, repositories own persistence, models define data shape, validators enforce boundaries, and middleware handles security and errors.

7. Repository Structure

```
backend/src/
├── config/          environment and database setup
├── controllers/     HTTP request handlers
├── errors/          safe application errors
├── middleware/      security, async, and 404 handling
├── models/          Mongoose schemas
├── repositories/    database access
├── routes/          API route definitions
├── services/        deployment, Docker, Nginx, and ports
├── utils/           small shared helpers
└── validators/      Zod validation and allowlists

frontend/src/
├── main.jsx         dashboard components and API calls
└── styles.css       responsive dashboard styling
```

8. Technology Stack

Layer	Choice	Reason
Backend	Node.js + JavaScript	Fast hackathon development and one ecosystem with React.
API	Express.js	Simple, familiar REST endpoints and middleware.
Container API	Dockerode	Controls Docker without shell interpolation.
Database	MongoDB + Mongoose	Flexible deployment documents and easy local Docker setup.
Frontend	React + Vite	Fast dashboard iteration and production build.
Proxy	Nginx	Routes public hostnames to allocated container ports.
Infrastructure	Linux + Docker Compose	Self-hosted and easy to reproduce.

> **Database decision:** Local MongoDB in Docker is the default for the hackathon. MongoDB Atlas remains optional through `MONGODB_URI`, avoiding live-IP allowlist problems during development.

9. Enhanced MVC Responsibilities

Layer	Responsibility
Routes	Map HTTP methods and paths to controller functions.
Controllers	Validate request data and return the correct HTTP response.
Services	Coordinate port reservation, Docker, Nginx, and state transitions.
Repositories	Keep MongoDB queries out of controllers and business logic.
Models	Define deployment fields, limits, enums, and indexes.
Validators	Reject unsafe branch names, images, URLs, and ports.
Middleware	Apply security headers, CORS, rate limits, sanitization, and error handling.

10. API Requirements

Method	Endpoint	Purpose
GET	`/api/health`	Service health check.
GET	`/api/deployments`	List recent deployments.
POST	`/api/deployments`	Create a preview deployment.
GET	`/api/deployments/:id`	Read deployment details.
GET	`/api/deployments/:id/stats`	Read CPU and memory statistics.
GET	`/api/deployments/:id/logs`	Read recent container logs.
DELETE	`/api/deployments/:id`	Stop and clean up a preview.

Example request

```
POST /api/deployments
Content-Type: application/json

{
  "branchName": "feature/header",
  "image": "nginx:alpine",
  "containerPort": 80
}
```

11. Security Design

- **Input validation:** Zod uses strict schemas and rejects unknown keys.
- **Branch safety:** branch names allow only bounded Git-like characters and length.
- **Image safety:** Docker images must be in `ALLOWED_IMAGES`; arbitrary image input is rejected.
- **Command safety:** Docker uses Dockerode; Nginx uses `execFile` with fixed command arguments.
- **MongoDB safety:** request-body keys beginning with `$` or containing `.` are removed.
- **HTTP security:** Helmet, restrictive CORS, body-size limits, and rate limiting are enabled.
- **Resource safety:** containers have CPU and memory limits.
- **Proxy safety:** route files use generated slugs and are validated before reload.
- **Secret safety:** environment files are ignored and never sent to the frontend.
- **Docker socket warning:** the Docker socket is root-equivalent and must never be publicly exposed.

12. Current MVP Boundary

> **Ready now:** dashboard deployment flow, MongoDB persistence, port reservation, mock/real Docker adapter, container metrics, logs endpoint, Nginx file generation, cleanup, local Compose setup, and security middleware.

> **Intentionally next:** signed GitHub/GitLab webhooks, branch checkout, Docker image building from a repository Dockerfile, real wildcard DNS/TLS, authentication, and a background job queue.

This boundary is useful in a presentation: the foundation is real and working, while the next phase is clearly planned rather than hidden.

13. Demo Plan for Judges

1. Start MongoDB and the backend in mock Docker mode.
2. Open the React dashboard.
3. Enter `feature/header` and click **Create preview**.
4. Show the status changing to running, the allocated port, and the generated preview URL.
5. Explain that the same adapter can use the real Docker socket by setting `USE_MOCK_DOCKER=false`.
6. Show the API response and dashboard metrics.
7. Create another branch to demonstrate a different port.
8. Delete the first preview and show that its port becomes available again.
9. Send an invalid branch or image and demonstrate the safe HTTP 400 response.

14. Presentation Talking Points

Slide	Message
Problem	Sharing feature branches requires manual infrastructure work.
Solution	DeployForge turns a branch into an isolated preview environment.
Architecture	React → Express → service layer → Docker/MongoDB/Nginx.
Security	Allowlist images, validate input, limit resources, and protect the Docker socket.
Demo	One branch request creates a tracked preview with a URL and metrics.
Impact	Faster reviews, less cloud lock-in, and fewer port-forwarding steps.
Future	Webhooks and automatic Docker builds make the flow fully zero-config.

One-minute explanation

> DeployForge is a self-hosted preview deployment engine. A developer submits a branch, and our Node.js backend validates it, reserves a port, starts an isolated Docker container through Dockerode, generates an Nginx route, and stores the deployment state in MongoDB. The React dashboard shows the branch, URL, status, CPU, memory, and actions. We use local MongoDB in Docker so the hackathon demo does not depend on changing Atlas IP allowlists. The system is secure by design: images are allowlisted, unsafe input is rejected, resource limits are applied, and the Docker socket is never exposed to the browser.

15. Setup and Verification

```
npm run backend:install
npm run frontend:install

# Start MongoDB
docker run -d --name deployforge-mongodb -p 27017:27017 mongo:7

# Terminal 1
npm run backend:dev

# Terminal 2
npm run frontend:dev

# Checks
npm run backend:check
npm run frontend:build
```

Open `http://localhost:5173`. The API runs on `http://localhost:4000`.

16. Roadmap

1. **Git provider integration:** verify signed push webhooks.
2. **Repository worker:** clone/fetch a branch into a safe workspace.
3. **Build service:** build a Dockerfile with BuildKit and stream logs.
4. **Readiness checks:** wait for an application health endpoint.
5. **HTTPS:** wildcard DNS plus Certbot or Caddy-managed certificates.
6. **Authentication:** protect dashboard actions and deploy endpoints.
7. **Queue and cleanup:** support retries, cancellation, TTL expiry, and restart reconciliation.

17. Success Metrics

Metric	Target
Demo deployment time	Under 15 seconds in mock or cached-image mode.
Port conflicts	Zero.
Invalid input acceptance	Zero malicious branch/image inputs accepted.
Dashboard refresh	Within 5 seconds.
Cleanup	Container, route, database state, and port are released.
Build quality	Backend checks and frontend production build pass.

18. Final Project Statement

DeployForge turns every Git branch into a shareable, isolated, production-like preview environment without requiring developers to manually manage infrastructure. It combines a simple Node.js control plane, safe

Docker automation, MongoDB state, Nginx routing, and a focused React dashboard. The current MVP is small enough to understand, strong enough to demonstrate, and structured so that Git webhooks and automatic repository builds can be added without rewriting the core architecture.